

Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	9/30/26	Parker Archuleta	Initial software design for Draw It or Lose It
2.0	10/4/26	Parker Archuleta	Updated Evaluation for Project 2
3.0	10/18/26	Parker Archuleta	Updated recommendations for Project 3

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room currently offers Draw It or Lose It as an Android application and wants to expand the game into a web-based application that can serve users on multiple operating systems and device types. The proposed solution is a client-server web application in which browsers provide the user interface while server-side Java components maintain authoritative game data and enforce the game rules. This approach reduces the need to create and maintain separate native applications for every platform and gives the client one central place to manage game sessions, teams, players, timing, and future persistence.

The prototype should organize the core game data using a shared Entity base class, Game, Team, Player, and a GameService manager. The Entity class provides the common id and name attributes. Game contains teams, Team contains players, and GameService maintains the collection of games. The GameService class should use the Singleton design pattern so only one service instance exists in memory in the running prototype. The add and lookup methods should use iteration to search existing objects before creating new ones. This prevents duplicate names and supports the client requirement that users be able to determine whether a requested name is already in use.

For a production web deployment, the in-memory prototype will need to be extended with persistent storage, authentication, secure network communication, concurrency controls, and reliable hosting. The design should keep core business logic on the server and use standards-based web technologies on clients. This allows The Gaming Room to support Windows, macOS, Linux, Android, and iOS browsers while keeping game state consistent and reducing platform-specific development effort.

Requirements

The following business and technical requirements define the first version of the application design:

- Each game must support one or more teams.
- Each team must support multiple players.
- Game names must be unique so a user can determine whether a game name is already in use.
- Team names must be unique within a game; player names should also be checked before a player is added to a team.
- Every Game, Team, and Player must have a unique identifier in addition to a name.
- Only one GameService instance should exist in application memory during execution, implemented with the Singleton pattern.
- The application should use iteration to search game, team, and player collections before adding new objects.
- The web version must serve multiple client platforms through a distributed client-server architecture.
- The design must support secure communication and a future path to persistent, scalable storage.

Design Constraints

- Cross-platform compatibility: The application must function through modern browsers on desktop and mobile operating systems. Client features should therefore rely on standards-based HTML, CSS, and JavaScript rather than operating-system-specific APIs whenever possible.
- Distributed state management: Multiple users can participate in the same game from separate devices. The server must be the authoritative source for shared game state so that teams, guesses, timers, and round status do not diverge between clients.
- Uniqueness and identity: Game, team, and player objects need stable identifiers, while user-facing names must be checked for duplicates. The prototype can use incrementing identifiers and iterator-based searches, but a production system should enforce uniqueness in persistent storage as well.
- Single service instance in the prototype: The GameService Singleton prevents separate in-memory service objects from maintaining conflicting game lists. In a scaled distributed deployment, this design must be supplemented by shared persistence because separate server processes cannot share a Java Singleton across machines.
- Concurrency: Two users may attempt to create the same name or update a game at nearly the same time. Production code must use atomic database constraints or transactions and server-side synchronization where appropriate instead of relying only on a search-then-add sequence.
- Network reliability and latency: The game includes timed rounds and frequent visual updates. Clients may experience slow connections or temporary outages, so timing should be based on server state and clients should be able to resynchronize after reconnecting.
- Security and privacy: User sessions and application traffic must be protected with HTTPS/TLS, server-side input validation, authentication where required, least-privilege access, and secure handling of stored credentials or user information.
- Stock image delivery: The game displays a large library of drawing assets. Images should be optimized and delivered efficiently, potentially through caching or a content delivery network, without embedding the entire library in every client.
- Maintainability: The codebase should separate presentation, business logic, and data access concerns. This reduces duplication, supports testing, and makes future platform or storage changes less disruptive.

System Architecture View

No additional response is required for this project section. For future development, the logical architecture would be described as browser-based clients communicating over HTTPS with a server-side application tier, which in turn communicates with persistent data storage and image-asset storage.

Domain Model

The UML diagram represents the core object model for Draw It or Lose It. At the top level, ProgramDriver provides the main() method used to run and test the application, while SingletonTester verifies that the same GameService instance is returned every time. ProgramDriver uses the service classes but does not represent a business entity in the game itself.

GameService is the central manager for Game objects. It stores a List<Game>, maintains the next identifiers for games, teams, and players, and exposes methods such as getInstance(), addGame(), getGame(), getGameCount(), getNextTeamId(), and getNextPlayerId(). The zero-to-many relationship between GameService and Game indicates that the service can manage multiple games. The Singleton

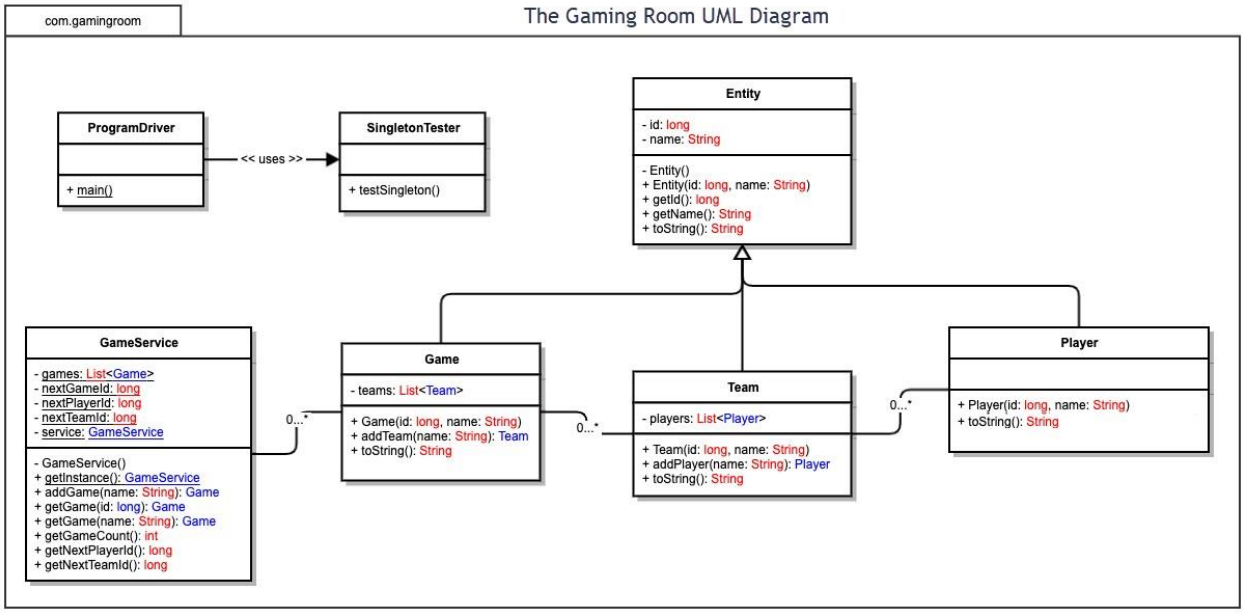
pattern is demonstrated by the private or controlled construction of GameService together with a static service instance and getInstance() method. This fulfills the requirement that only one GameService object exist in memory in the prototype.

Entity is the superclass for Game, Team, and Player. It contains the common id and name attributes, constructors, accessor methods, and toString() behavior. Using a base class demonstrates inheritance and abstraction: shared identity behavior is written once in Entity and reused by the three specialized classes. This reduces duplicate code and ensures that each game object follows the same identity structure.

Game extends Entity and owns a List<Team>. Its addTeam() method searches the existing teams before creating a new Team, so duplicate team names can be prevented. Team also extends Entity and owns a List<Player>. Its addPlayer() method performs the same type of search before adding a Player. Player is the simplest subclass and inherits its identifier and name from Entity. These zero-to-many associations model the required hierarchy: one Game can contain many Team objects, and one Team can contain many Player objects.

The model demonstrates encapsulation because each class controls access to its internal fields and collections through methods rather than exposing implementation details directly. It demonstrates inheritance through Entity, abstraction through the shared Entity concept, and polymorphism because Game, Team, and Player can all be treated as Entity objects when only their common id, name, or string representation is needed. The object relationships also use composition or aggregation to represent the natural containment of teams within games and players within teams.

The iterator pattern supports the uniqueness requirements. GameService iterates through games in addGame() and getGame(); Game iterates through teams in addTeam(); and Team iterates through players in addPlayer(). When a requested name already exists, the method can return the existing object instead of creating a duplicate. Combined with the unique numeric identifiers issued by GameService, this gives the prototype predictable identity and lookup behavior while keeping the implementation simple and readable.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	macOS can host a Java-based web application because it supports the Java Development Kit, common web frameworks, databases, and other UNIX-based server technologies. However, macOS is not commonly selected for large-scale production web hosting because Apple no longer provides a dedicated macOS Server product and macOS generally must run on Apple hardware. This makes scaling to many servers less flexible and potentially more expensive than using Linux-based cloud or commodity server infrastructure. There is not normally a separate macOS server operating-system license because macOS is included with Apple hardware, but the required Apple hardware can increase infrastructure costs. For Draw It or Lose It, macOS would be suitable for development and testing but would be a weaker choice for hosting thousands of concurrent players	Linux is an excellent platform for hosting the server-side portion of Draw It or Lose It. It is widely used for web servers, cloud infrastructure, Java applications, databases, containers, and distributed systems. The application could be deployed using a Java application server or framework along with technologies such as Nginx, Apache, Docker, and a relational database. Linux can also scale horizontally by adding additional virtual machines or containers behind a load balancer as the number of players increases. A major advantage is cost because many Linux distributions can be used without purchasing a per-server operating-system license, although optional commercial support subscriptions are available. The primary disadvantage is that system administration may require developers or administrators	Windows Server can effectively host the Draw It or Lose It web application and supports Java, IIS, databases, containers, PowerShell automation, and enterprise networking technologies. It is especially useful for organizations that already rely on Microsoft infrastructure such as Active Directory or other Microsoft services. Windows Server can support a scalable deployment, including multiple application servers behind a load balancer. Its main disadvantage is cost because Windows Server is commercially licensed, typically according to server processor cores, and some deployments may also require Client Access Licenses. These expenses can increase as the server environment grows. Windows is therefore technically capable but may cost more to operate than Linux for a large distributed web application	Android and iOS are not appropriate platforms for hosting the production Draw It or Lose It server. Mobile operating systems are designed for end-user devices rather than continuously available public web services. Battery limitations, background-process restrictions, changing network connections, application sandboxing, and device availability make phones and tablets unsuitable for maintaining authoritative game sessions for thousands of users. Instead, Android and iOS devices should operate as clients that communicate with a centralized server over HTTPS. Because mobile devices are not being used as servers, the client would not incur mobile server operating-system licensing costs.

Client Side	Mac users can access Draw It or Lose It through a modern web browser, so CTS does not need to develop a separate native macOS application. The front end should use standards-based HTML, CSS, and JavaScript so that the same interface can operate in Safari, Chrome, Firefox, and other supported browsers. Development time and cost are reduced because most client code can be shared with Windows, Linux, Android, and iOS. However, additional testing time is required to identify browser-specific differences, particularly in Safari. Developers need web-development and responsive-design expertise rather than specialized native macOS programming skills.	Linux users can access the application through modern browsers such as Firefox and Chromium-based browsers. Because Draw It or Lose It will use a browser-based interface, CTS does not need to maintain a separate Linux desktop application. This reduces development cost and allows the same HTML, CSS, and JavaScript codebase to serve multiple operating systems. The team must still test browser compatibility and avoid relying on proprietary operating-system features, fonts, media formats, or browser-specific functionality. Supporting Linux therefore adds testing work but should not require a separate development team.	Windows clients can run Draw It or Lose It through browsers such as Microsoft Edge, Chrome, and Firefox. Using a responsive web application avoids the time and expense of creating a separate Windows-native application. The development team must test different browsers, screen resolutions, display-scaling settings, keyboard navigation, and responsive layouts. Because the same standards-based web technologies are used across platforms, a single front-end team can support Windows alongside Linux and macOS rather than maintaining an independent Windows development team.	Supporting Android and iOS requires the greatest amount of client-side compatibility testing because phones and tablets vary significantly in screen size, pixel density, orientation, browser behavior, touch input, processing capability, and network reliability. CTS should use responsive HTML, CSS, and JavaScript so that most of the same front-end code can be reused across desktop and mobile clients. Developers should test the application in both Android Chrome and iOS Safari as well as on representative physical devices and emulators. This increases testing time and requires expertise in responsive and mobile web design, but it remains less expensive than maintaining independent native Android and iOS applications.

Development Tools	Development on macOS can use Java with the JDK and IDEs such as IntelliJ IDEA, Eclipse, or Visual Studio Code. Front-end development can use HTML, CSS, JavaScript or TypeScript, Node.js tooling, Git, and browser developer tools. Docker and other development utilities are also available. Most required tools have free or open-source versions, although commercial editions such as IntelliJ IDEA Ultimate may require paid licenses. A Mac is also necessary if CTS later decides to build or extensively test native iOS software with Xcode. For the planned browser-based application, however, one cross-platform development team can perform most work without needing a dedicated macOS development group.	Linux provides strong support for Java development using the JDK, Maven or Gradle, IntelliJ IDEA, Eclipse, Visual Studio Code, Git, Docker, Nginx or Apache, and relational databases such as PostgreSQL. Many of these tools are free or open source, which reduces development and deployment costs. Linux is also well suited for shell scripting, build automation, continuous integration, containers, and cloud deployment. The development team should include members familiar with Linux server administration, but a separate Linux-only programming team is not necessary because the Java back end and browser front end are largely portable across operating systems.	Windows supports the JDK, Maven or Gradle, IntelliJ IDEA, Eclipse, Visual Studio Code, Git, PowerShell, Docker, IIS, and major database systems. Windows Subsystem for Linux can also give developers access to Linux-style command-line tools when necessary. Many development tools are available without licensing fees, although commercial IDE editions, Windows Server, or certain Microsoft development products may introduce additional costs. Because Java and web technologies are cross-platform, CTS would not need a separate Windows development team. Developers mainly need to account for Windows-specific deployment, administration, and browser testing.	Because the first version of Draw It or Lose It will be delivered through a responsive web browser, mobile development can primarily use the same HTML, CSS, JavaScript or TypeScript, Java back end, Git, and web-development tools used for desktop clients. Browser developer tools, responsive-design modes, Android emulators, and iOS simulators can be used during testing. If native applications are developed in the future, Android Studio with Kotlin or Java and Xcode with Swift could be required. Android Studio is available without a license fee, while Apple development may involve Apple hardware and developer-program costs. For the current web-based design, separate Android and iOS development teams are unnecessary; a cross-platform web team with mobile testing expertise should be sufficient.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: CTS recommends Linux as the production server operating platform for Draw It or Lose It. Linux provides strong support for Java, web servers, relational databases, containers, cloud infrastructure, automation, and horizontal scaling without adding operating-system licensing costs. The browser-based client should remain platform-neutral so Windows, macOS, Linux, Android, and iOS users can participate without separate native applications.

2. **Operating Systems Architectures**: A 64-bit Linux distribution on x86-64 or ARM64 cloud/server hardware is appropriate. Linux uses a monolithic kernel with modular support for device drivers and other kernel components. The kernel manages hardware resources, processes, memory, and file system operations, while user-space applications such as the Java server interact with these services through system calls. The application should use a layered client-server architecture: the presentation layer runs in the browser, the application layer runs the Java game service and APIs, and the data layer stores persistent game and user data. This separation allows each tier to be secured, tested, and scaled independently. Containers can further standardize the runtime environment across development, testing, and production.

3. **Storage Management**: Use a relational database such as PostgreSQL for structured data including users, games, teams, players, round state, and audit information. Primary keys should provide durable identifiers and unique constraints should enforce names where required. The drawing library should be stored as files or object-storage assets rather than large database records, with the database retaining metadata and references to those assets. Backups, replication, and retention policies should be configured for production. Linux supports file systems such as ext4 and XFS, which organize files into directories and use metadata to manage storage locations, permissions, and access. Using one of these file systems for server storage, alongside PostgreSQL for structured game data, would provide reliable data organization and support for the application's storage needs.

4. **Memory Management**: Linux provides virtual memory, process isolation, paging, and kernel-managed memory allocation. The Java Virtual Machine adds automatic garbage collection for application objects. Draw It or Lose It should keep only active session data and frequently used resources in memory, release references when games end, and avoid loading the entire drawing library at once. Application memory limits and JVM heap settings can be monitored and tuned as usage grows. The Singleton GameService controls one in-process service object, but persistent shared state should not depend on a single process remaining alive.

5. **Distributed Systems and Networks**: Clients should communicate with the server through HTTPS-based web APIs, and near-real-time events can use WebSocket connections or controlled polling. The server should own the official game timer and state so all clients can resynchronize after latency or a temporary disconnect. A load balancer can distribute traffic across multiple application instances, while a shared database or cache keeps data consistent. The design must account for DNS, connectivity, server outages, database availability, and retries. Health checks, logging, monitoring, graceful reconnection, and redundant services reduce the impact of individual failures.

6. **Security**: All browser-to-server traffic should use HTTPS with modern TLS. Authentication sessions should use secure, HttpOnly, SameSite cookies or another well-designed token strategy. Passwords, if stored, must be salted and hashed with an appropriate password-hashing algorithm rather than encrypted or stored in plain text. Server code should validate all input, authorize access to game resources, apply least privilege to database and service accounts, protect secrets outside source code, log security-relevant events, and keep dependencies patched. Rate limiting and protections against common web attacks such as injection, cross-site scripting, cross-site request forgery, and session abuse should be included in production hardening.